
Pooling Bias Meets Attention: Heterogeneous Panel Forecasting with Lightweight Adapters

Yu Wang*

Xiangchen Liu†

Abstract

Time series foundation models (TSFMs) share parameters across all units in a panel, implicitly assuming that every unit responds identically to common covariates. When units differ in covariate sensitivity—a phenomenon known in econometrics as coefficient heterogeneity—pooled estimation produces systematic forecast bias that amplifies with covariate magnitude [1]. We connect this classical result to the TSFM setting and propose a lightweight output-correction adapter that attaches to a frozen backbone. Guided by the form of the bias, the adapter takes a proxy for each unit’s covariate sensitivity together with future regressor values and produces an additive correction to the base forecast. On synthetic heterogeneous panel data, the adapter reduces test RMSE by 15–26%, with the largest gains on high-sensitivity units and short-context regimes where the backbone cannot implicitly adapt. Ablations confirm that the adapter’s value scales with the severity of heterogeneity and diminishes as context length increases, consistent with the theoretical prediction.

1 Introduction

Time series foundation models (TSFMs) have achieved strong zero-shot and few-shot forecasting performance across diverse domains [3, 4, 5]. A common deployment scenario involves *panel data*: many related units (customers, sensors, regions) observed over time with shared exogenous covariates. TSFMs process each unit through shared parameters, producing forecasts that implicitly assume all units respond identically to the same covariate.

This assumption is often violated. In credit risk, different borrowers react differently to interest rate changes. In energy forecasting, buildings vary in temperature sensitivity. In retail, stores differ in price elasticity. The econometrics literature has long recognized this as *coefficient heterogeneity*: when units differ in their sensitivity to common regressors but are estimated via pooled regression, the resulting estimator is inconsistent [1], and the forecast bias amplifies when covariates take extreme values. Bonhomme & Manresa [2] proposed grouped fixed-effects estimators for this setting, but their methods assume access to the full estimation pipeline rather than a frozen pretrained model.

Recent work has begun integrating covariates into TSFMs. CoRA [6] injects covariates via learned causal embeddings into frozen TSFMs. ChronosX [7] fine-tunes adapter layers for covariate integration. TFMAdapter [8] cascades univariate TSFM forecasts with covariates via instance-level nonparametric learning. On the adaptation side, δ -Adapter [9] proposes input and output correction placements for frozen forecasters, and TS-Memory [10] distills retrieval-based corrections into a lightweight adapter. However, all of these treat covariate response as homogeneous—the same transformation is applied regardless of which unit is being forecast.

*Capital One. The views expressed are solely those of the author and do not represent the views of Capital One or any of its affiliates. Contact: yuwang9908@gmail.com

†Department of Family and Consumer Sciences, California State University, Long Beach. xiangchen.liu@csulb.edu.

We bridge the econometrics literature on coefficient heterogeneity with the TSFM adaptation literature. Our contributions are: (1) we connect the classical pooling bias result to the TSFM setting and demonstrate empirically that frozen TSFMs exhibit the predicted bias amplification pattern on heterogeneous panel data; (2) we propose a lightweight output-correction adapter whose design is directly motivated by the form of the bias, taking a proxy for each unit’s covariate sensitivity and producing an additive forecast correction; (3) we show through ablations that the adapter’s value scales with heterogeneity severity and diminishes with context length, consistent with theoretical predictions.

2 Method: Correcting Homogeneous Coefficients

Consider a panel $\{y_{it}, \mathbf{x}_t\}$ with N cross-sectional units and T time periods. The data-generating process for unit i is

$$y_{it} = f(y_{i,t-1}) + \mu_i + \beta_i^\top \mathbf{x}_t + \varepsilon_{it}, \quad (1)$$

where f is a common transition function, μ_i is an individual fixed effect, $\mathbf{x}_t$ is a vector of common regressors, and β_i is a unit-specific slope coefficient. A TSFM shares all parameters across units, with no mechanism conditioned on unit identity. Its mapping from $\mathbf{x}_t$ to forecast is therefore a single pooled function, implicitly constraining β_i to a common β . The resulting forecast bias for unit i is $(\beta_i - \beta)^\top \mathbf{x}_t$: units more sensitive than average are systematically under-predicted, units less sensitive are over-predicted, and the error amplifies when the regressors take extreme values [1].

2.1 Output-Correction Adapter

The bias formula $(\beta_i - \beta)^\top \mathbf{x}_t$ directly suggests the form of the correction: it is a function of the unit’s sensitivity β_i and the regressor values $\mathbf{x}_t$. If we knew β_i exactly, we could compute the bias in closed form and subtract it. In practice β_i is unknown, so we replace it with a proxy $\mathbf{s}_i$ estimated from data and let a small MLP learn the mapping from $(\mathbf{s}_i, \mathbf{x}_t)$ to the correction. Given a frozen backbone producing base forecast $\hat{y}_{it}^{\text{base}}$, the corrected forecast is

$$\hat{y}_{it} = \hat{y}_{it}^{\text{base}} + g(\mathbf{s}_i, \mathbf{x}_t), \quad t = T + 1, \dots, T + H, \quad (2)$$

where g is a lightweight MLP and $\mathbf{x}_t$ is the regressor vector at time t . We place the correction at the output of the frozen backbone rather than the input, because an input-side correction would be diluted through multiple frozen layers before reaching the forecast.

We consider two ways to construct $\mathbf{s}_i$. Variant A sets $\mathbf{s}_i$ to a vector of known attributes of unit i , such as its industry, location, or risk category. Variant B requires no such attributes: it sets $\mathbf{s}_i = \frac{1}{T_c} \sum_{\tau=1}^{T_c} y_{i\tau} \cdot \mathbf{x}_\tau$, which measures how strongly unit i ’s outcome co-moves with each regressor over the context window.

2.2 Training Procedure

A naive approach—training the adapter on the same data as the backbone—does not work. The backbone already predicts its training data well, so there is little systematic bias left for the adapter to learn from. On unseen data, however, the pooling bias emerges: high-sensitivity units are systematically under-predicted and low-sensitivity units are over-predicted. The adapter must be trained on this kind of data to learn a useful correction.

We therefore split the time axis into four consecutive segments. The backbone trains end-to-end on the first 50% of the time series. We then freeze the backbone and train only the adapter on the next 20%—data the backbone has never seen, where its forecasts exhibit systematic pooling bias. The adapter learns to map each unit’s $\mathbf{s}_i$ and regressor values to the correction that reduces the gap between the backbone’s forecast and the true outcome. The next 15% serves as validation for checkpoint selection, and the final 15% is held out for testing.

3 Experiments with Synthetic Datasets

3.1 Experiment Setup

To isolate the effect of coefficient heterogeneity under controlled conditions, we conduct a Monte Carlo experiment in which the true β_i is known by construction. Specifically, we generate a panel of $N=2000$ units over $T=512$ time steps with 3 common regressors. Units are divided into three equal groups with sensitivity vectors $\beta \in \{[0.1, 0.1, 0.2], [1.5, 1.0, 1.5], [5.0, 3.0, 5.0]\}$. The concrete DGP instantiating Equation 1 is:

$$y_{it} = \phi y_{i,t-1} + \gamma \sin(y_{i,t-1}) + \mu_i + \delta_i \cdot \frac{t}{T} + s_t + \beta_i^\top \mathbf{x}_t + \varepsilon_{it}. \quad (3)$$

The transition function f combines a linear autoregressive term ($\phi=0.6$) with a nonlinear sinusoidal term ($\gamma=0.3$), both shared across units. Each unit has its own fixed effect μ_i and trend slope δ_i . The seasonal component $s_t = \sum_k A_k \sin(2\pi t/P_k + \varphi_k)$ combines three shared frequencies with periods $P \in \{12, 24, 52\}$. The three covariates each follow an independent AR(1) process with persistence $\rho \in \{0.3, 0.5, 0.7\}$, and the idiosyncratic noise is $\varepsilon_{it} \sim \mathcal{N}(0, 0.3^2)$. The nonlinear dynamics, trends, and seasonality are included to give the forecasting task enough complexity to justify a transformer backbone—a linear model cannot capture these patterns, but the heterogeneity in β_i remains the only source of unit-level variation that the pooled backbone cannot resolve.

We use a patch-based transformer as the backbone, with 4 layers, embedding dimension 128, 4 attention heads, and 869K total parameters, following the lightweight TSFM design of TTM [11]. The context length is 32 and the forecast horizon is 12. The adapter is a two-layer MLP with 3,340 parameters, less than 0.4% of the backbone. We choose this architecture to demonstrate the heterogeneity problem in a controlled setting; the adapter itself is model-agnostic and can attach to any frozen backbone, including production-scale TSFMs.

3.2 Main Results: the Adapter Reduces RMSE Significantly

Under pooled estimation, Pesaran & Smith [1] show that the estimator is inconsistent when slope coefficients are heterogeneous, with forecast errors concentrating on units whose β_i deviates most from $\bar{\beta}$ and amplifying with the covariate magnitude $|\mathbf{x}_t|$. Table 1 is consistent with the first implication: the base TSFM’s RMSE ranges from 1.49 for the low-sensitivity group to 20.99 for the high-sensitivity group, a $14\times$ gap.

Figure 1 is consistent with the second: in the left panel, MAE for high- β units increases sharply with $|\mathbf{x}_t|$, while low- β units remain stable, producing a fan-shaped pattern across groups. The right panels show that both adapter variants compress this fan-out, reducing the slope of the high- β curve and producing more uniform error across groups and covariate magnitudes. The oracle adapter, which has access to the true β_i , achieves the flattest profile.

Table 1: Test RMSE by model and sensitivity group.

Model	RMSE	G0 (low)	G1 (med)	G2 (high)	Impr.
Base TSFM	12.72	1.49	6.47	20.99	—
+ Static (A)	9.94	1.17	5.13	16.38	−21.8%
+ Inferred (B)	10.84	1.31	5.50	17.91	−14.7%
+ Oracle	9.42	1.32	5.04	15.47	−25.9%

Note: G0/G1/G2 correspond to low/medium/high β groups. Improvement is relative to Base TSFM.

The adapter substantially reduces this bias. The overall RMSE drops by 15–26% depending on the variant, with the largest gains on the high-sensitivity group (G2: $20.99 \rightarrow 15.47$ – 17.91). The ordering—Oracle best, then Static, then Inferred, then Base—is expected: the more the adapter knows about a unit’s true β_i , the better it can correct. Variant B is notable because it achieves 14.7% improvement without any metadata, estimating unit sensitivity purely from the context window. Together, these results confirm that the pooling bias is both real and correctable with a lightweight, post-hoc adapter.

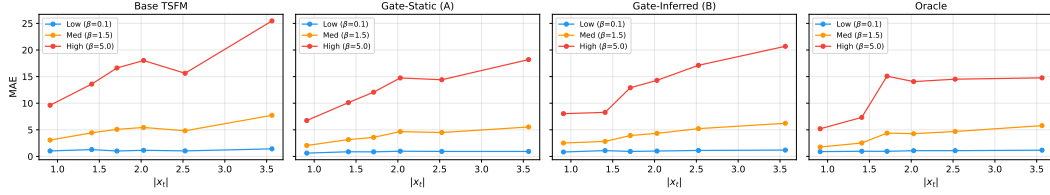


Figure 1: MAE vs. covariate magnitude $|x_t|$ by sensitivity group.

Note: Left panel shows the base TSFM; right panels show each adapter variant. High- β group error grows with $|x_t|$ under the base model; adapters compress this fan-out.

3.3 Ablations: Context Length and Heterogeneity Level

The magnitude of the pooling bias $(\beta_i - \bar{\beta})^\top x_t$ is governed by two factors: the dispersion of β_i across units, which determines how far each unit’s sensitivity lies from the pooled estimate, and the context length available to the backbone, which determines whether attention can implicitly narrow this gap by leveraging longer histories. We ablate each factor separately, re-running the full pipeline—backbone training, adapter training, and evaluation—each time.

Table 2: Ablation results.

(a) Context length.					(b) Heterogeneity level.				
Context	Base	Static	Inferred		Level	β spread	Base	Static	Inferred
16	10.87	−15.8%	−18.9%		Low	[0.8, 1.0, 1.2]	4.65	−19.3%	−26.1%
32	12.72	−14.8%	−18.0%		Med	[0.5, 1.5, 3.0]	8.30	−25.5%	−24.2%
64	11.04	−5.0%	−8.6%		High	[0.1, 1.5, 5.0]	12.72	−18.8%	−17.6%
96	9.88	+8.4%	+0.6%						

Note: Static and Inferred columns show % RMSE change relative to the base TSFM. Negative means the adapter helps.

Table 2a varies the context length from 16 to 96 steps. The adapter’s improvement decreases monotonically: at context 16–32, RMSE drops by 15–19%; at 64, gains shrink to 5–9%; at 96, the adapter provides no benefit and slightly hurts for the static variant. This pattern is consistent with the hypothesis that longer context allows the backbone to implicitly infer unit-level sensitivity through attention, reducing the residual bias available for the adapter to correct. The practical implication is that the adapter is most valuable in cold-start and limited-history scenarios—precisely the regimes where panel forecasting is hardest.

Table 2b varies the dispersion of β_i across three levels, from near-homogeneous ([0.8, 1.0, 1.2]) to highly dispersed ([0.1, 1.5, 5.0]). The adapter improves RMSE by 17–26% at every level, indicating that even moderate heterogeneity produces enough pooling bias for the correction to be worthwhile.

4 Discussion

We have shown that coefficient heterogeneity, a well-understood phenomenon in panel econometrics, produces systematic forecast bias when TSFMs pool parameters across units, and that a lightweight output-correction adapter can substantially mitigate this bias without modifying the backbone. The adapter’s design follows directly from the structure of the pooling bias: it takes a proxy for each unit’s covariate sensitivity and the future regressor values, and learns an additive correction.

A key finding is the interaction between context length and adapter value. With sufficient context, transformer attention can implicitly infer unit-level sensitivity from the history, reducing the residual bias and leaving less for the adapter to correct. In short-context regimes, this implicit adaptation fails, and the adapter becomes essential. This has practical implications for cold-start and limited-history scenarios common in finance, healthcare, and IoT applications, where units often have too little history for the backbone to adapt on its own.

Our experiments are limited to synthetic data, where the heterogeneity structure is known by construction. Validation on real-world panel datasets with heterogeneous covariate effects—such as credit risk

portfolios with interest rate sensitivity or building energy panels with temperature sensitivity—would strengthen the conclusions. We also plan to integrate the adapter with production-scale TSFMs such as Chronos and Moirai, and to investigate theoretical consistency guarantees for the correction.

References

- [1] M. H. Pesaran and R. Smith. Estimating long-run relationships from dynamic heterogeneous panels. *Journal of Econometrics*, 68(1):79–113, 1995.
- [2] S. Bonhomme and E. Manresa. Grouped patterns of heterogeneity in panel models. *Econometrica*, 83(3):1147–1184, 2015.
- [3] A. Ansari et al. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, 2024.
- [4] A. Das et al. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024.
- [5] G. Woo et al. Unified training of universal time series forecasting transformers. In *ICML*, 2024.
- [6] Y. Qin et al. CoRA: Covariate-aware adaptation for time series foundation models. In *ICLR*, 2026.
- [7] S. Arango et al. ChronosX: Adapting pretrained time series models with exogenous variables. *arXiv preprint*, 2025.
- [8] A. Dange and S. Sarawagi. TFMAadapter: Lightweight instance-level adaptation of foundation models for forecasting with covariates. In *CIKM*, 2025.
- [9] Y. Zhang et al. The forecast after the forecast: A post-processing shift in time series. *arXiv preprint arXiv:2601.20280*, 2025.
- [10] Z. Shi et al. TS-Memory: Plug-and-play memory for time series foundation models. *arXiv preprint arXiv:2602.11550*, 2025.
- [11] V. Ekambaram et al. TTMs: Fast multi-level tiny time mixers for improved zero-shot and few-shot forecasting of multivariate time series. In *ICML*, 2024.
- [12] Y. Mundlak. On the pooling of time series and cross section data. *Econometrica*, 46(1):69–85, 1978.

Online Appendix

A Experiment Details

A.1 Data-Generating Process

Table 3 lists all parameters of the synthetic DGP used in the main experiment (Section 3). The heterogeneity ablation (Table 2b) uses the same DGP with modified β vectors as shown in Table 4.

A.2 Backbone Architecture

The backbone is a patch-based transformer encoder. Table 5 lists the architectural and training hyperparameters. Input patches are constructed by sliding a window of length 3 with stride 3 over the context, yielding non-overlapping patches. Each target patch is projected to d_{model} via a linear layer. Covariate patches are formed by concatenating the 3 covariates within each patch (yielding dimension $3 \times 3 = 9$) and projected separately. The two embeddings are merged additively with a learnable positional encoding, then passed through a pre-norm transformer encoder. The prediction head takes the last patch representation and maps it to the forecast horizon through a two-layer MLP with GELU activation.

Table 3: DGP parameters for the main experiment.

Parameter	Description	Value
N	Number of units	2000
T	Time steps	512
C	Number of common regressors	3
K	Number of groups	3
β_{G0}	Low-sensitivity group	[0.1, 0.1, 0.2]
β_{G1}	Medium-sensitivity group	[1.5, 1.0, 1.5]
β_{G2}	High-sensitivity group	[5.0, 3.0, 5.0]
σ_β	Within-group β noise std	0.1
ϕ	AR(1) coefficient (shared)	0.6
γ	Nonlinear sin coefficient (shared)	0.3
σ_μ	Fixed effect std	0.5
σ_δ	Trend slope std	0.005
P	Seasonal periods	{12, 24, 52}
A	Seasonal amplitudes	{0.5, 0.3, 0.2}
ρ	Covariate AR(1) persistence	{0.3, 0.5, 0.7}
σ_u	Covariate innovation std	1.0
σ_ε	Idiosyncratic noise std	0.3
σ_z	Static feature noise std	0.1
Seed	Random seed	42

Table 4: β vectors for the heterogeneity sweep ablation.

Level	G0	G1	G2
Low	[0.8, 0.3, 0.8]	[1.0, 1.0, 1.5]	[1.2, 2.0, 3.0]
Med	[0.5, 0.3, 0.8]	[1.5, 1.0, 1.5]	[3.0, 2.0, 3.0]
High	[0.1, 0.3, 0.8]	[1.5, 1.0, 1.5]	[5.0, 2.0, 3.0]

A.3 Adapter Architecture

The adapter is a two-layer MLP with GELU activation. Its input is the concatenation of unit features $\mathbf{s}_i$ and the flattened future regressors $\mathbf{x}_{T+1:T+H}$; its output is an additive correction of dimension H . Table 6 lists the details for each variant.

A.4 Context Length Ablation

For the context length sweep (Table 2a), we train a separate backbone for each context length in $\{16, 32, 64, 96\}$, keeping all other hyperparameters identical. The number of patches varies accordingly: $\lfloor (\text{context} - 3)/3 \rfloor + 1$, yielding 5, 10, 21, and 32 patches respectively. The sliding window stride for dataset construction is set to 12 for all context lengths.

Table 5: Backbone hyperparameters.

Parameter	Description	Value
<i>Architecture</i>		
d_{model}	Embedding dimension	128
n_{heads}	Attention heads	4
n_{layers}	Transformer layers	4
d_{ff}	Feedforward dimension	512
Patch length		3
Patch stride		3
Dropout		0.1
Context length		32
Forecast horizon		12
Total parameters		869K
<i>Training</i>		
Optimizer		AdamW
Weight decay		10^{-4}
Learning rate		2×10^{-4}
LR schedule		Cosine annealing
Batch size		256
Max epochs		200
Early stopping patience		20
Gradient clipping		1.0
Training split		[0%, 50%)

Table 6: Adapter hyperparameters by variant.

	Static (A)	Inferred (B)	Oracle
<i>Architecture</i>			
$\mathbf{s}_i$ definition	$\mathbf{z}_i$ (noisy one-hot)	$\frac{1}{T_c} \sum_{\tau} y_{i\tau} \cdot \mathbf{x}_{\tau}$	True β_i
$\mathbf{s}_i$ dimension	3	3	3
Input dimension	$3 + 12 \times 3 = 39$	$3 + 36 = 39$	$3 + 36 = 39$
Hidden dimension	64	64	64
Output dimension	12	12	12
Activation	GELU	GELU	GELU
Parameters	3,340	3,340	3,340
<i>Training</i>			
Optimizer	AdamW (weight decay = 10^{-4})		
Learning rate	10^{-3} (constant)		
Batch size	256		
Max epochs	200		
Early stopping patience	20		
Training split	[50%, 70%)		

Note: The Oracle variant uses the true β_i as $\mathbf{s}_i$ and serves as an upper bound on adapter performance. It is not a practical method.